



UPLA

Construcción de software

Semana 04 (Laboratorio)

Aplicación de estándares en el
desarrollo del Sistema Construsoft
Ticket

ASIGNATURA
Construcción de software

ACTIVIDAD 01

“El software es una gran combinación entre arte e ingeniería.”

—Bill Gates

1. Objetivo

Aplicar estándares de construcción de software en un caso práctico mediante la implementación funcional del módulo de registro de tickets en el sistema Construsoft Ticket, integrando arquitectura, lógica de negocio y persistencia de datos.

2. Contexto

En la semana anterior, el equipo de desarrollo logró definir la estructura base del sistema Construsoft Ticket, estableciendo una arquitectura organizada y alineada a buenas prácticas.

Sin embargo, el sistema aún no cumple su propósito principal: gestionar información real.

Actualmente, el registro de tickets solo existe a nivel de interfaz o lógica básica, pero no se almacena en ningún sistema persistente, lo que limita su utilidad.

Para resolver este problema, el equipo debe avanzar hacia una implementación más completa, integrando:

- Persistencia de datos
- Conexión entre capas

- Validación de información
- Flujo completo de registro

Tal como señalan Martin Fowler y Robert C. Martin, la calidad del software no solo depende de su estructura, sino de su correcta implementación funcional.

3. Problema de la semana

¿Cómo implementar un módulo funcional de registro de tickets que permita almacenar información en una base de datos, manteniendo una arquitectura limpia y aplicando estándares de construcción de software?

4. Condiciones y restricciones

El estudiante debe considerar:

- a. Uso del stack definido:
 - C# (.NET)
 - React + Vite
 - PostgreSQL
- b. Mantener la arquitectura de capas.
- c. Aplicar principios SOLID
- d. Implementar persistencia de datos.
- e. Garantizar que el flujo funcione de extremo a extremo:
 - Frontend → API → Base de datos

5. Actividades / Preguntas orientadoras

El estudiante deberá:

- a) ¿Cómo integrará PostgreSQL con el backend del sistema?

- b) ¿Qué cambios debe realizar en la capa Infrastructure?
- c) ¿Cómo se relaciona la entidad Ticket con la base de datos?
- d) ¿Cómo se implementará la persistencia (guardar datos)?
- e) ¿Cómo validar que los datos se registran correctamente?
- f) ¿Cómo conectar el frontend con el backend?
- g) ¿Qué mejoras aplicaría al formulario del sistema?
- h) ¿Qué estándares y principios se mantienen en esta etapa?

6. Entregable

El estudiante deberá presentar:

1. Backend con persistencia de datos implementada.
2. Base de datos configurada (PostgreSQL)
3. Registro funcional de tickets (almacenamiento real)
4. Frontend conectado con API
5. Evidencia de funcionamiento (capturas o video)
6. Documento breve justificando:
 - Integración de capas
 - Uso de base de datos
 - Estándares aplicados

7. Reflexión final

- ¿Qué diferencia existe entre un sistema sin persistencia y uno con base de datos?
- ¿Qué retos encontró al integrar múltiples capas del sistema?
- ¿Cómo influye la persistencia en la calidad del software?

Solución (modelo)

Solución modelo: Inicio del desarrollo del sistema Construsoft Ticket

1. Propósito de la actividad

Implementar el registro funcional de tickets en el sistema Construsoft Ticket, integrando:

- Persistencia en PostgreSQL;
- Acceso a datos en la capa Infrastructure;
- Contratos en Application;
- Exposición del endpoint en API;
- Consumo básico desde React.

Esta semana corrige una decisión que no era compatible con Clean Architecture: usar ApplicationDbContext directamente desde la capa Application. La solución correcta será usar un repositorio definido como interfaz en Application e implementado en Infrastructure.

2. Antes de comenzar: corrección de estructura

2.1 Error que no debe mantenerse

Si en el archivo:

```
backend/ConstrusoftTicket.Application/Services/TicketService.cs
```

se colocó algo como:

```
using ConstrusoftTicket.Infrastructure.Data;
```

eso debe corregirse, porque:

- Application no debe depender de Infrastructure;
- DbContext pertenece a Infrastructure;
- La capa Application solo debe conocer interfaces.

3. Archivos que deben eliminarse o corregirse

3.1 Archivo que debe corregirse

Ruta:

backend/ConstruSoftTicket.Application/Services/TicketService.cs

Acción:

- No eliminar el archivo
- Sí reemplazar completamente su contenido por la versión correcta que se mostrará más adelante.

3.2 Línea que debe eliminarse si existe

En el archivo:

backend/ConstruSoftTicket.Application/Services/TicketService.cs

si existe esta línea:

using ConstruSoftTicket.Infrastructure.Data;

debe eliminarse.

3.3. Archivos que pueden eliminarse si fueron creados incorrectamente

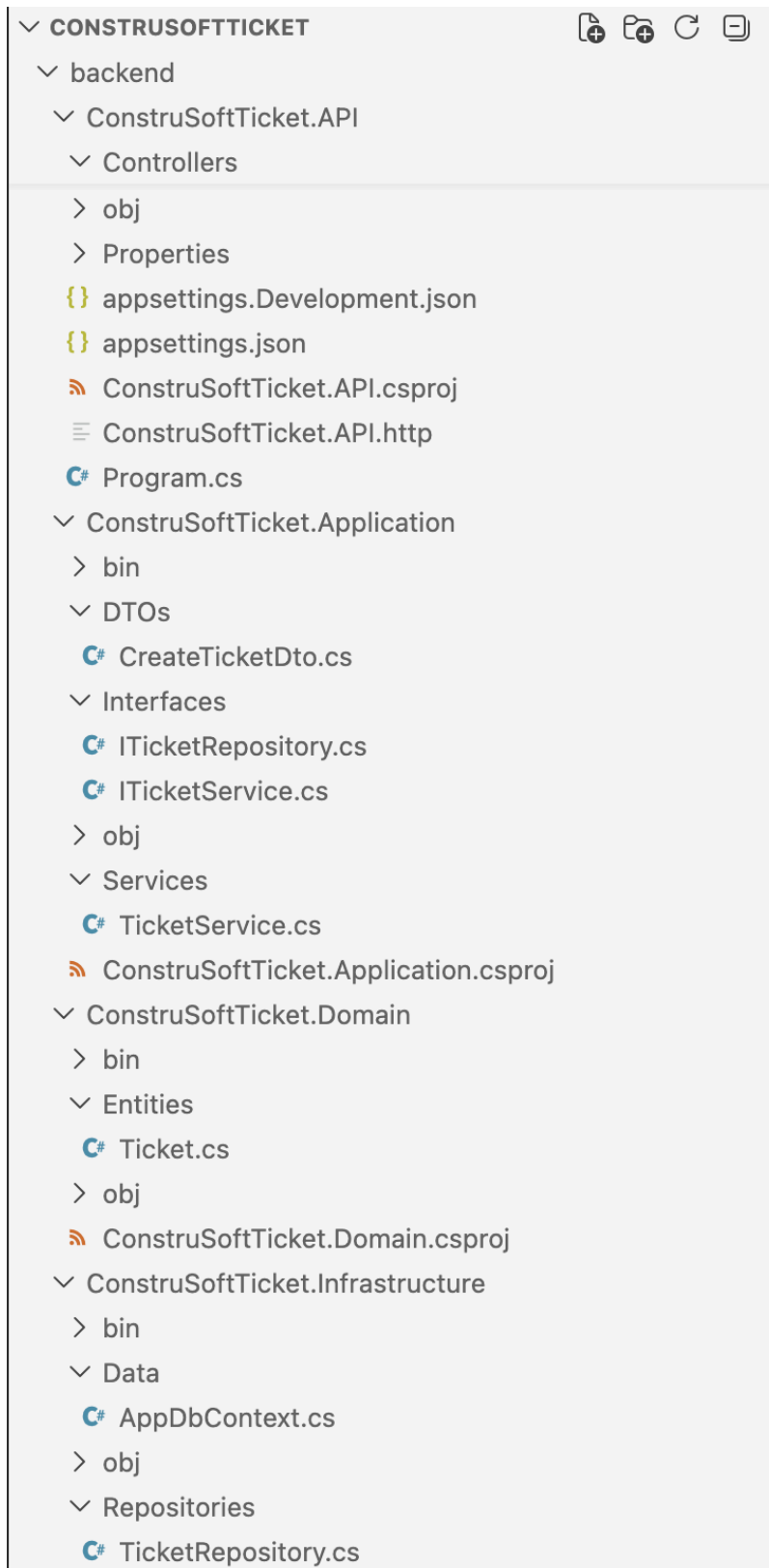
Si durante pruebas previas se creó algún archivo experimental fuera de la estructura correcta, por ejemplo:

- Repositorios dentro de Application con acceso a DbContext;
- Clases temporales duplicadas;
- Servicios que dependan directamente de Infrastructure;

deben eliminarse.

4. Estructura final esperada del backend

Al terminar esta semana, el backend debe quedar así:



5. Desarrollo paso a paso

Paso 1. Verificar paquetes en Infrastructure

Objetivo

Asegurar que Infrastructure tenga las dependencias necesarias para EF Core y PostgreSQL.

Ubicación desde donde se ejecuta

Entrar a:

backend/ConstruSoftTicket.Infrastructure

Comandos

macOS	Windows 11
cd backend/ConstruSoftTicket.Infrastructure	cd backend\ConstruSoftTicket.Infrastructure

Luego ejecutar:

```
dotnet add package Microsoft.EntityFrameworkCore --version 8.0.6
```

```
dotnet add package Npgsql.EntityFrameworkCore.PostgreSQL --version 8.0.4
```

```
dotnet add package Microsoft.EntityFrameworkCore.Tools --version 8.0.6
```

Resultado esperado

El archivo:

backend/ConstruSoftTicket.Infrastructure/ConstruSoftTicket.Infrastructure.csproj

debe contener referencias similares a estas:

```
7
8      <ItemGroup>
9        <PackageReference Include="Microsoft.EntityFrameworkCore" Version="8.0.6" />
10       <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="8.0.6">
11         <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
12         <PrivateAssets>all</PrivateAssets>
13       </PackageReference>
14       <PackageReference Include="Npgsql.EntityFrameworkCore.PostgreSQL" Version="8.0.4" />
15     </ItemGroup>
```

Paso 2. Crear o verificar la entidad Ticket

Ruta exacta

backend/ConstruSoftTicket.Domain/Entities/Ticket.cs

Si la carpeta Entities no existe

Crearla manualmente en VS Code.

Contenido final del archivo

```
C# Ticket.cs ×
backend > ConstrusoftTicket.Domain > Entities > C# Ticket.cs > { } ConstrusoftTicket.Domain.Entities
1 namespace ConstrusoftTicket.Domain.Entities;
2
3     5 referencias
4 public class Ticket
5 {
6     1 referencia
7     | public Guid Id { get; set; }
8     1 referencia
9     | public string Titulo { get; set; } = string.Empty;
10    1 referencia
11    | public string Descripcion { get; set; } = string.Empty;
12    1 referencia
13    | public DateTime FechaCreacion { get; set; }
14    1 referencia
15    | public string Estado { get; set; } = string.Empty;
16 }
```

Paso 3. Crear o verificar el DTO

Ruta exacta

backend/ConstrusoftTicket.Application/DTOs/CreateTicketDto.cs

Si la carpeta Entities no existe

Crearla manualmente en VS Code.

Contenido final del archivo

```
C# CreateTicketDto.cs ×
backend > ConstrusoftTicket.Application > DTOs > C# CreateTicketDto.cs > { } ConstrusoftTicket.Application.DTOs
1 namespace ConstrusoftTicket.Application.DTOs;
2
3     3 referencias
4 public class CreateTicketDto
5 {
6     1 referencia
7     | public string Titulo { get; set; } = string.Empty;
8     1 referencia
9     | public string Descripcion { get; set; } = string.Empty;
10 }

```

Paso 4. Crear o verificar la interfaz del servicio

Ruta exacta

backend/ConstruSoftTicket.Application/Interfaces/ITicketService.cs

Si la carpeta Interfaces no existe

Crearla manualmente.

Contenido final

```
C# ITicketService.cs X
backend > ConstruSoftTicket.Application > Interfaces > C# ITicketService.cs > { } ConstruSoftTicket
1  using ConstruSoftTicket.Application.DTOS;
2
3  namespace ConstruSoftTicket.Application.Interfaces;
4
5  4 referencias
6  public interface ITicketService
7  {
8      2 referencias
9      void CrearTicket(CreateTicketDto dto);
10 }
```

Paso 5. Crear la interfaz del repositorio

Ruta exacta

backend/ConstruSoftTicket.Application/Interfaces/ITicketRepository.cs

Contenido final

```
C# ITicketRepository.cs X
backend > ConstruSoftTicket.Application > Interfaces > C# ITicketRepository.cs > { } ConstruSoftTic
1  using ConstruSoftTicket.Domain.Entities;
2
3  namespace ConstruSoftTicket.Application.Interfaces;
4
5  4 referencias
6  public interface ITicketRepository
7  {
8      2 referencias
9      void Add(Ticket ticket);
10 }
```

Resultado esperado

Ahora Application define el contrato del repositorio, pero no conoce cómo se guarda en la base de datos.

Paso 6. Corregir el servicio TicketService

Ruta exacta

backend/ConstruSoftTicket.Application/Services/TicketService.cs

Acción

Reemplazar completamente el contenido por este:

```
C# TicketService.cs X
backend > ConstruSoftTicket.Application > Services > C# TicketService.cs > { } ConstruSoftTicket.A

1  using ConstruSoftTicket.Application.DTOS;
2  using ConstruSoftTicket.Application.Interfaces;
3  using ConstruSoftTicket.Domain.Entities;
4
5  namespace ConstruSoftTicket.Application.Services;
6
7  2 referencias
8  public class TicketService : ITicketService
9  {
10     2 referencias
11     private readonly ITicketRepository repository;
12
13     0 referencias
14     public TicketService(ITicketRepository repository)
15     {
16         this.repository = repository;
17     }
18
19     2 referencias
20     public void CrearTicket(CreateTicketDto dto)
21     {
22         var ticket = new Ticket
23         {
24             Id = Guid.NewGuid(),
25             Titulo = dto.Titulo,
26             Descripcion = dto.Descripcion,
27             FechaCreacion = DateTime.UtcNow,
28             Estado = "Abierto"
29         };
30
31         repository.Add(ticket);
32     }
33 }
```

Importante

En este archivo no debe quedar ninguna referencia a:

`using ConstrusoftTicket.Infrastructure.Data;`

Paso 7. Crear el DbContext en Infrastructure

Ruta exacta

`backend/ConstrusoftTicket.Infrastructure/Data/AppDbContext.cs`

Si la carpeta Data no existe

Crearla manualmente.

Contenido final

```
C# AppDbContext.cs X
backend > ConstrusoftTicket.Infrastructure > Data > C# AppDbContext.cs > { } ConstrusoftTicket.Infrastructure

1  using ConstrusoftTicket.Domain.Entities;
2  using Microsoft.EntityFrameworkCore;
3
4  namespace ConstrusoftTicket.Infrastructure.Data;
5
6  4 referencias
7  public class AppDbContext : DbContext
8  {
9      0 referencias
10     public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
11     {
12     }
13
14     1 referencia
15     public DbSet<Ticket> Tickets => Set<Ticket>();
16 }
```

Paso 8. Crear el repositorio concreto en Infrastructure

Ruta exacta

`backend/ConstrusoftTicket.Infrastructure/Repositories/TicketRepository.cs`

Si la carpeta Repositories no existe

Crearla manualmente.

Contenido final

```
C# TicketRepository.cs X
kend > ConstrusoftTicket.Infrastructure > Repositories > C# TicketRepository.cs > { } Con

1  using ConstrusoftTicket.Application.Interfaces;
2  using ConstrusoftTicket.Domain.Entities;
3  using ConstrusoftTicket.Infrastructure.Data;
4
5  namespace ConstrusoftTicket.Infrastructure.Repositories;
6
7  2 referencias
   public class TicketRepository : ITicketRepository
8  {
9      3 referencias
       private readonly AppDbContext context;
10
11     0 referencias
       public TicketRepository(AppDbContext context)
12     {
13         this.context = context;
14     }
15
16     2 referencias
       public void Add(Ticket ticket)
17     {
18         context.Tickets.Add(ticket);
19         context.SaveChanges();
20     }
21 }
```

Paso 9. Verificar referencias entre proyectos

Objetivo

Asegurar que los proyectos tengan las referencias necesarias para compilar.

Ubicación desde donde se ejecuta

Entrar a:

backend

Comandos

macOS	Windows 11
cd backend	cd backend

Luego ejecutar:

```
dotnet add ConstrusoftTicket.Application/ConstrusoftTicket.Application.csproj reference  
ConstrusoftTicket.Domain/ConstrusoftTicket.Domain.csproj
```

```
dotnet add ConstrusoftTicket.Infrastructure/ConstrusoftTicket.Infrastructure.csproj  
reference ConstrusoftTicket.Application/ConstrusoftTicket.Application.csproj
```

```
dotnet add ConstrusoftTicket.Infrastructure/ConstrusoftTicket.Infrastructure.csproj  
reference ConstrusoftTicket.Domain/ConstrusoftTicket.Domain.csproj
```

```
dotnet add ConstrusoftTicket.API/ConstrusoftTicket.API.csproj reference  
ConstrusoftTicket.Application/ConstrusoftTicket.Application.csproj
```

```
dotnet add ConstrusoftTicket.API/ConstrusoftTicket.API.csproj reference  
ConstrusoftTicket.Infrastructure/ConstrusoftTicket.Infrastructure.csproj
```

Resultado esperado

- Application referencia a Domain
- Infrastructure referencia a Application y Domain
- API referencia a Application e Infrastructure

Paso 10. Configurar la cadena de conexión

Ruta exacta

backend/ConstrusoftTicket.API/appsettings.json

Acción

Agregar o ajustar la sección ConnectionStrings.

Contenido sugerido

```
{} appsettings.json ×
backend > ConstrusoftTicket.API > {} appsettings.json > ...
1  {
2    "ConnectionStrings": {
3      "DefaultConnection": "Host=localhost;Port=5432;Database=ConstrusoftTicketDB;Username=postgres;Password=1234"
4    },
5    "Logging": {
6      "LogLevel": {
7        "Default": "Information",
8        "Microsoft.AspNetCore": "Warning"
9      }
10   },
11   "AllowedHosts": "*"
12 }
```

Importante

La contraseña 1234 es solo un ejemplo. Debe reemplazarse por la contraseña real configurada en PostgreSQL.

Paso 11. Corregir Program.cs

Ruta exacta

backend/ConstrusoftTicket.API/Program.cs

Acción

Reemplazar completamente el contenido del archivo por esta versión, para evitar inconsistencias con cambios previos:


```
1 using ConstruSoftTicket.Application.Interfaces;
2 using ConstruSoftTicket.Application.Services;
3 using ConstruSoftTicket.Infrastructure.Data;
4 using ConstruSoftTicket.Infrastructure.Repositories;
5 using Microsoft.EntityFrameworkCore;
6
7 var builder = WebApplication.CreateBuilder(args);
8
9 // Add services to the container.
10 builder.Services.AddControllers();
11 builder.Services.AddEndpointsApiExplorer();
12 builder.Services.AddSwaggerGen();
13
14 builder.Services.AddDbContext<AppDbContext>(options =>
15     options.UseNpgsql(builder.Configuration.GetConnectionString("DefaultConnection")));
16
17 builder.Services.AddScoped<ITicketRepository, TicketRepository>();
18 builder.Services.AddScoped<ITicketService, TicketService>();
19
20 builder.Services.AddCors(options =>
21 {
22     options.AddPolicy("AllowFrontend", policy =>
23     {
24         policy.AllowAnyOrigin()
25             .AllowAnyHeader()
26             .AllowAnyMethod();
27     });
28 });
29
30 var app = builder.Build();
31 // Configure the HTTP request pipeline.
32 if (app.Environment.IsDevelopment())
33 {
34     app.UseSwagger();
35     app.UseSwaggerUI();
36 }
37
38 app.UseCors("AllowFrontend");
39 app.UseHttpsRedirection();
40 app.UseAuthorization();
41 app.MapControllers();
42 app.Run();
```

Paso 12. Crear o verificar el controlador

Ruta exacta

backend/ConstruSoftTicket.API/Controllers/TicketController.cs

Si la carpeta Controllers no existe

Crearla manualmente.

Contenido final

```
C# TicketController.cs X
backend > ConstrusoftTicket.API > Controllers > C# TicketController.cs > { } ConstrusoftTicket.A

1  using ConstrusoftTicket.Application.DTOS;
2  using ConstrusoftTicket.Application.Interfaces;
3  using Microsoft.AspNetCore.Mvc;
4
5  namespace ConstrusoftTicket.API.Controllers;
6
7  [ApiController]
8  [Route("api/[controller]")]
   1 referencia
9  public class TicketController : ControllerBase
10 {
   2 referencias
11     private readonly ITicketService ticketService;
12
   0 referencias
13     public TicketController(ITicketService ticketService)
14     {
15         this.ticketService = ticketService;
16     }
17
18     [HttpPost]
   0 referencias
19     public IActionResult Crear([FromBody] CreateTicketDto dto)
20     {
21         ticketService.CrearTicket(dto);
22         return Ok(new { mensaje = "Ticket registrado correctamente." });
23     }
24 }
```

Paso 13. Verificar si PostgreSQL está listo

Objetivo

Confirmar que existe la base de datos ConstrusoftTicketDB.

Opción terminal

macOS	Windows 11
psql postgres o psql -U postgres	psql -U postgres

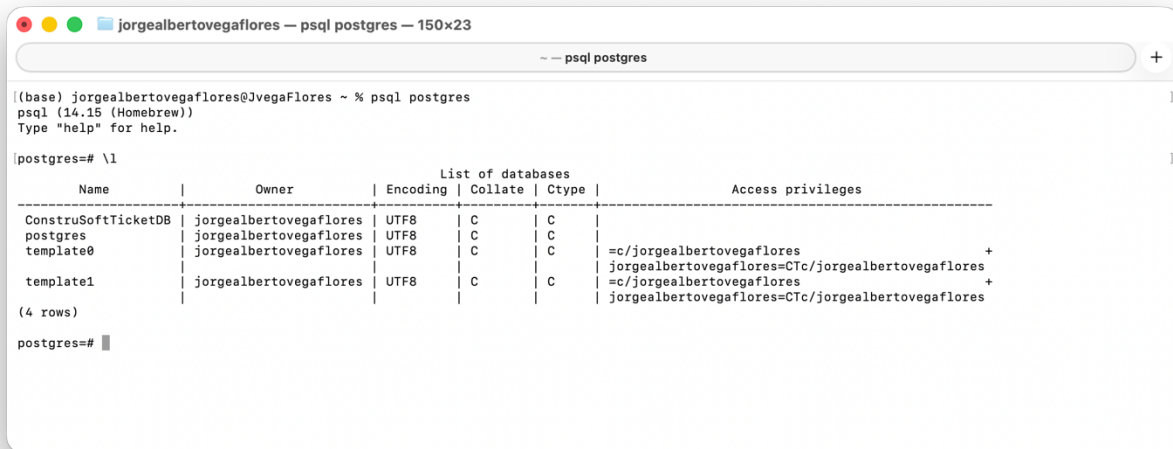
Luego ejecutar:

`\l`

Resultado esperado

Debe aparecer la base de datos:

ConstruSoftTicketDB



```
(base) jorgealbertovegaflores@JvegaFlores ~ % psql postgres
psql (14.15 (Homebrew))
Type "help" for help.

postgres=# \l
               List of databases
  Name          | Owner          | Encoding | Collate | Ctype | Access privileges
-----+-----+-----+-----+-----+-----
ConstruSoftTicketDB | jorgealbertovegaflores | UTF8     | C       | C     |
postgres         | jorgealbertovegaflores | UTF8     | C       | C     | =c/jorgealbertovegaflores+
template0        | jorgealbertovegaflores | UTF8     | C       | C     | jorgealbertovegaflores=CTC/jorgealbertovegaflores+
template1        | jorgealbertovegaflores | UTF8     | C       | C     | =c/jorgealbertovegaflores+
                |                   |          |         |       | jorgealbertovegaflores=CTC/jorgealbertovegaflores
(4 rows)

postgres=#
```

Si no existe

Crear con:

`CREATE DATABASE "ConstruSoftTicketDB";`

Salir con:

`\q`

Paso 14. Crear la migración inicial

Ubicación desde donde se ejecuta

Entrar a:

backend/ConstruSoftTicket.API

Comandos

macOS	Windows 11
<code>cd backend/ConstruSoftTicket.API</code>	<code>cd backend\ConstruSoftTicket.API</code>

Luego ejecutar:

```
dotnet ef migrations add InitialCreate --project ../ConstruSoftTicket.Infrastructure --startup-project .
```

En caso de error de referencia instalar en backend/ConstruSoftTicket.API

Comando

```
dotnet add package Microsoft.EntityFrameworkCore.Design --version 8.0.6
```

Resultado esperado

Se crea una carpeta Migrations dentro del proyecto Infrastructure.

Paso 15. Aplicar la migración a la base de datos.

Desde la misma carpeta

backend/ConstruSoftTicket.API

Comando

```
dotnet ef database update --project ../ConstruSoftTicket.Infrastructure --startup-project .
```

Resultado esperado

La tabla Tickets queda creada en ConstruSoftTicketDB.

Paso 16. Ejecutar la API

Ubicación exacta

backend/ConstruSoftTicket.API

Comandos

macOS	Windows 11
dotnet run	dotnet run

Resultado esperado

La terminal mostrará una URL similar a:

```
(base) jorgealbertovegaflares@JvegaFlores ConstrusoftTicket.API % dotnet run
Compilando...
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5153
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: /Users/jorgealbertovegaflares/Documents/ConstrusoftTicket/backend/ConstrusoftTicket.API
warn: Microsoft.AspNetCore.HttpsPolicy.HttpsRedirectionMiddleware[3]
      Failed to determine the https port for redirect.
-
```

Paso 17. Probar en Swagger

Acción

Abrir en navegador:

<https://localhost:XXXX/swagger>

Endpoint a probar

POST /api/ticket

JSON de prueba

```
{
  "titulo": "PC no enciende",
  "descripcion": "Equipo del laboratorio 3 no responde al encendido."
}
```

Resultado esperado

Respuesta:

```
{
  "mensaje": "Ticket registrado correctamente."
}
```

Paso 18. Verificar en PostgreSQL

Comando de acceso

macOS	Windows 11
psql -U postgres -d ConstrusoftTicketDB	psql -U postgres -d ConstrusoftTicketDB

Consulta

SELECT * FROM "Tickets";

Resultado esperado

Debe aparecer al menos un ticket insertado.

Paso 19. Conectar el frontend con el backend

Ruta exacta

frontend/src/pages/CreateTicket.jsx

Acción

Reemplazar completamente el contenido por esta versión:

```
CreateTicket.jsx ×
frontend > src > pages > CreateTicket.jsx > CreateTicket > handleSubmit

1  import { useState } from "react";
2
3  export default function CreateTicket() {
4    const [titulo, setTitulo] = useState("");
5    const [descripcion, setDescripcion] = useState("");
6
7    const handleSubmit = async (event) => {
8      event.preventDefault();
9
10     const response = await fetch("https://localhost:7208/api/ticket", {
11       method: "POST",
12       headers: {
13         "Content-Type": "application/json"
14       },
15       body: JSON.stringify({
16         titulo,
17         descripcion
18       })
19     });
20
21     if (response.ok) {
22       alert("Ticket registrado correctamente");
23       setTitulo("");
24       setDescripcion("");
25     } else {
26       alert("Error al registrar el ticket");
27     }
28   };
29
```

```

30     return (
31       <div>
32         <h1>Registro de Ticket</h1>
33
34         <form onSubmit={handleSubmit}>
35           <div>
36             <label>Título</label>
37             <input
38               type="text"
39               value={titulo}
40               onChange={(event) => setTitulo(event.target.value)}
41             />
42           </div>
43
44           <div>
45             <label>Descripción</label>
46             <textarea
47               value={descripcion}
48               onChange={(event) => setDescription(event.target.value)}
49             />
50           </div>
51
52           <button type="submit">Registrar ticket</button>
53         </form>
54       </div>
55     );
56   }

```

Importante

Si el puerto del backend no es 7208, debe reemplazarse por el puerto real mostrado por dotnet run.

Paso 20. Verificar App.jsx

Ruta exacta

frontend/src/App.jsx

Contenido esperado

```
App.jsx ×
frontend > src > App.jsx > default
1  import CreateTicket from "../pages/CreateTicket";
2
3  function App() {
4    |   return <CreateTicket />;
5  }
6
7  export default App;
```

Paso 21. Ejecutar el frontend

Ubicación exacta

frontend

Comandos

macOS	Windows 11
cd frontend	cd frontend
npm install	npm install
npm run dev	npm run dev

Resultado esperado

El navegador mostrará la pantalla de registro de tickets.

6. Archivos que deben quedar finalmente en el proyecto

Backend

Domain

- backend/ConstruSoftTicket.Domain/Entities/Ticket.cs

Application

- backend/ConstruSoftTicket.Application/DTOs/CreateTicketDto.cs
- backend/ConstruSoftTicket.Application/Interfaces/ITicketRepository.cs
- backend/ConstruSoftTicket.Application/Interfaces/ITicketService.cs

- backend/ConstruSoftTicket.Application/Services/TicketService.cs

Infrastructure

- backend/ConstruSoftTicket.Infrastructure/Data/AppDbContext.cs
- backend/ConstruSoftTicket.Infrastructure/Repositories/TicketRepository.cs
- carpeta Migrations generada por EF Core
- archivo .csproj con paquetes correctos

API

- backend/ConstruSoftTicket.API/Controllers/TicketController.cs
- backend/ConstruSoftTicket.API/Program.cs
- backend/ConstruSoftTicket.API/appsettings.json

Frontend

- frontend/src/pages/CreateTicket.jsx
- frontend/src/App.jsx

7. Archivos que no deben quedar con contenido incorrecto

No debe quedar en TicketService.cs

- Ninguna referencia a ConstruSoftTicket.Infrastructure.Data
- Ninguna dependencia directa a AppDbContext

No debe haber repositorios dentro de Application

que implementen acceso real a la base de datos.

8. Criterios de verificación docente

Elemento	Verificación
Arquitectura	Application no depende de Infrastructure
Persistencia	Infrastructure contiene AppDbContext y TicketRepository
API	expone POST /api/ticket
Base de datos	tabla Tickets creada
Frontend	envía datos al backend
Flujo	datos se guardan realmente en PostgreSQL

